

Relatório: Dívida técnica, SOLID e refatoração

Projeto: Portal de Aluguel de Carros (MVP) — ``ai-driven-class/avaliacaoFinal``
Escopo: API Express/TypeScript (``sourceCode``) e frontend React/Vite (``frontend``)
Data: 24 de março de 2026

1. Pré-análise (resumo do diagnóstico original)

1.1 Contexto arquitetural

- Backend em camadas: rotas → controllers → services → repositories, com composição manual em ``server.ts``.
- Frontend com estado e efeitos concentrados em ``Home.tsx`` e integração HTTP em um único módulo ``api.ts``.

1.2 Acoplamento excessivo identificado

Área	Problema
Middleware de auth	<code>`auth.middleware`</code> importava <code>`AuthPayload`</code> de <code>`auth.service`</code> , acoplando infraestrutura HTTP ao módulo de aplicação.
Repositórios	Implementações importavam mocks diretamente, sem contrato explícito para inversão de dependência.
<code>`Home.tsx`</code>	Orquestração de sessão, navegação, catálogo, mutações e efeitos no mesmo componente.
<code>`api.ts`</code>	Cliente HTTP, parsing, <code>`localStorage`</code> e todas as chamadas em um único arquivo.

1.3 Violações / riscos SOLID

- **S:** ``BookingService`` concentrava validação temporal, regras de negócio, cálculo de valor e persistência; ``BookingController`` misturava validação HTTP com regras de visibilidade para cliente.
- **O:** Evolução de persistência ou políticas de reserva exigia editar classes concretas sem extensão por portas.
- **L:** Ausência de contratos formais para repositórios dificultava substituição segura (ex.: passagem para SQL).
- **I:** Dependência de serviços “largos” aceitável no tamanho atual, porém propensa a crescer.
- **D:** Services dependiam de classes concretas de repositório, não de abstrações definidas pela aplicação.

2. Prompt de refatoração (solicitação do usuário)

> Agora execute as mudanças sugeridas e ao final gere um relatório completo em pdf, da pré análise anterior e dos ajustes realizados registrando os problemas encontrados, as técnicas aplicadas e o prompt de refatoração.

A partir desse pedido, foram implementadas as recomendações priorizadas na análise anterior: extração de tipos de autenticação, portas de repositório, fatiamento da camada de API no frontend, hooks para compor a página inicial, e extração de regras de data para o domínio.

3. Ajustes realizados

3.1 Backend

```
| Técnica | O que foi feito |
|-----|-----|
| **Separação de tipos / DIP na borda** | Criado `src/types/auth-payload.ts` com `AuthPayload`. `auth.middleware` e `auth.service` passam a depender desse módulo neutro. |
| **Ports & Adapters** | Interfaces em `src/ports/*-repository.port.ts` (`IUserRepository`, `ICustomerRepository`, `IVehicleRepository`, `IBookingRepository`). Repositórios em memória implementam essas interfaces. |
| **Injeção por construtor** | `AuthService`, `BookingService` e `VehicleService` recebem os portos (`I*Repository`) em vez de tipos concretos. |
| **Domínio (SRP / DRY)** | Módulo `src/domain/booking-dates.ts`: `todayISODate`, `inclusiveDayCount`, `parseCreateBookingDates`, `parseVehicleAvailabilityRange` (com discriminação `kind` para mensagens de erro), `parseBookingDateRange` genérico. |
| **Mensagens HTTP preservadas** | `VehicleService` mapeia `kind` (`missing` \ `format` \ `order`) para as mesmas mensagens de topo usadas antes da refatoração. |
```

Arquivos novos (backend):

`src/types/auth-payload.ts`, `src/ports/\*.port.ts`, `src/domain/booking-dates.ts`.

Arquivos alterados (backend):

repositórios (implements), `auth.service.ts`, `auth.middleware.ts`, `booking.service.ts`, `vehicle.service.ts`.

3.2 Frontend

```
| Técnica | O que foi feito |
|-----|-----|
| **Barrel + módulos por agregado** | Pasta `src/api/`: `config.ts`, `http-client.ts`, `json-utils.ts`, `parsers.ts`, `error-utils.ts`, `session-storage.ts`, `auth-api.ts`, `vehicles-api.ts`, `bookings-api.ts`, `index.ts` (reexportações). Removido o monólito `api.ts`. |
| **Hooks de composição** | `useAuthSession` (login, validação de token, logout, 401), `useFleetCatalog` (veículos, reservas, navegação de página, seleção de veículo),
```

``useBookingMutations` (criar/cancelar reserva). |`

`| **Tipo de navegação** | `AppPageId` movido para `src/types/AppPage.ts` para hooks não dependerem de componentes de UI. |`

Arquivos novos (frontend):

``src/api/*` (módulos listados), `src/hooks/useAuthSession.ts`, `useFleetCatalog.ts`,
`useBookingMutations.ts`, `src/types/AppPage.ts`.`

Arquivos alterados (frontend):

``Home.tsx` (apenas composição), `AppMenu.tsx` (import do tipo).`

4. Problemas encontrados durante a execução

1. **Ambiente de build:** em ``sourceCode``, ``tsc`` não estava disponível até ``npm install`` local — dependências do projeto precisam instaladas antes do build.
2. **Mensagens de erro de disponibilidade:** a primeira extração para funções genéricas de data alteraria textos da API; foi necessário introduzir ``parseVehicleAvailabilityRange`` com tipo de resultado discriminado (``VehicleAvailabilityParseResult``) para manter compatibilidade de mensagens.
3. **Acoplamento hook ↔ UI:** ``useFleetCatalog`` importava ``AppPageId`` de ``AppMenu``; resolvido extraindo o tipo para ``types/AppPage.ts``.

5. Técnicas aplicadas (lista consolidada)

- **Dependency Inversion:** portas (``I*Repository``) consumidas pelos serviços; implementações concretas na camada de infraestrutura em memória.
- **Separation of Concerns:** middleware não depende mais do pacote de serviço só para tipo; domínio de datas isolado.
- **Single Responsibility:** módulos de API no cliente separados por responsabilidade (HTTP, sessão, parsing, erros, endpoints).
- **Composition root:** ``server.ts`` continua instanciando implementações concretas e injetando nos serviços (adequado ao tamanho do MVP).
- **Custom hooks (React):** estado e efeitos distribuídos de forma testável e reutilizável.

6. Validação

- ``npm run build`` em ``sourceCode``: **sucesso** (TypeScript + cópia do OpenAPI).
- ``npm run build`` em ``frontend``: **sucesso** (``tsc`` + Vite).

7. Próximos passos sugeridos (fora do escopo desta entrega)

- Implementação ``Sql*Repository`` atrás das mesmas portas, com testes de contrato.
- Testes unitários para ``domain/booking-dates.ts`` e para hooks com MSW ou mocks de ``api``.
- Container de DI leve (ex.: ``awilix`` ou `factory functions`) se o grafo de dependências crescer.

*Documento gerado para acompanhamento acadêmico / auditoria de refatoração. Versão Markdown
fonte: ``docs/RELATORIO_REFATORACAO_SOLID.md``.*